Struct Memory-R1: Intermediate Implementation Report

Yifan Liu **Liam Gallagher** **David Courtis** **Jiazhou Liang**

1 Problem Definition

We consider an LLM-based agent augmented with an external memory system that persists across interactions:

$$\mathcal{A} = (\pi_\theta,\ \mathcal{X},\ \mathcal{Y},\ \mathcal{M},\ \mathcal{E}),$$

- **policy** π_θ: maps an observation and a memory state to a distribution over action trajectories;
- **input (observation) space** $\mathcal{X}$: the set of all possible user queries, dialogue contexts, or environmental observations the agent may receive;
- **output (response) space** $\mathcal{Y}$: the set of all possible natural-language responses the agent may produce;
- **memory space** $\mathcal{M}$: the set of all possible external memory states the agent can read from and, optionally, write to;
- **external environment interface** $\mathcal{E}$: the set of tools, APIs, or retrieval services through which the agent can perform actions beyond pure language generation (e.g., a memory retrieval server, a search engine, or a code executor).

An *interaction* is a temporally ordered sequence of turns between a user and the agent:

$$\mathcal{I} = (t_1, t_2, \ldots, t_T),$$

where T is the total number of turns in the interaction. At turn t ($1 \le t \le T$), the following exchange occurs:

1. The user provides an input $x_t \in \mathcal{X}$ (a query, instruction, or follow-up).
2. The agent observes x_t together with the current memory state $M_t \in \mathcal{M}$.
3. The agent generates an action trajectory as a sequence of *language actions* (reasoning steps, final answers) and *environment actions* (memory queries, tool calls). The trajectory length L_t may vary per turn.
4. The agent emits a final response $y_t \in \mathcal{Y}$ extracted from τ_t.
5. The memory state may be updated: $M_{t+1} = f_{\text{update}}(M_t, \tau_t)$, where f_{update} encodes any write operations performed during the turn.

The objective is to learn a policy π_θ that controls memory access when and how memory should be queried to improve downstream task performance.

2 Struct Memory-R1

Recent work, notably Search-R1 [Jin et al., 2025] and Memory-R1 [Yan et al., 2025][1], has shown that RL can effectively train agents to manage flat external memory banks. A *flat memory state* is an unordered collection of N memory entries:

$$M_t^{\text{flat}} = \{m_1, m_2, \ldots, m_N\}, \quad m_i = (\text{key}_i,\ \text{attributes}_i),$$

where each entry m_i consists of a textual key, a textual attribute, and optional metadata. However, flat representations treat all memory entries as independent, discarding the hierarchical and relational structure often present in real-world user state (e.g., a dietary preference nested under a user profile, or a hotel booking linked to a specific trip). We hypothesize that exposing this structure to the policy enables more selective retrieval and reduces over-retrieval of irrelevant entries. Therefore, we proposed *Struct Memory-R1*.

[1] Memory-R1 does not provide a public codebase. We built a publicly available implementation at `https://github.com/YifanLiu2/Struct_Memory_R1.git`.

Preprint. Under review.

2.1 Memory States

Building on Search-R1 and Memory-R1, our work extends reinforcement learning with verifiable rewards from external information retrieval to *structured* memory for long-context conversational AI scenarios. Specificly, a *structured memory state* at turn t is a rooted tree:

$$M_t = (V_t,\ E_t,\ r),$$

V_t is a finite set of **nodes** (memory units). Each node $v \in V_t$ carries:

- a **type label** $\ell(v) \in \mathcal{L}$ (e.g., ITINERARY, DAY, POI, SPEAKER, FACT);
- a set of **textual attributes** $\text{attr}(v) = \{(k_1, val_1), \ldots, (k_m, val_m)\}$.

$E_t \subseteq V_t \times V_t$ is a set of directed **parent–child edges** encoding hierarchical relations such as:

$$\text{ITINERARY} \rightarrow \text{VERSION} \rightarrow \text{DAY} \rightarrow \text{POI},$$

$r \in V_t$ is the designated **root node**.

2.2 Action Space

The agent's action space is $\mathcal{A} = \mathcal{A}_{\text{lang}} \cup \mathcal{A}_{\text{env}}$, where: $\mathcal{A}_{\text{lang}}$: **Reasoning** (`<think>`) and **Answer** (`<answer>`) generation. And $\mathcal{A}_{\text{env}}$: **Memory Query** (`<memory>`q`</memory>`) and **Memory Update** (ADD, UPDATE, DELETE).

A memory query sends $q \in \mathcal{Q}$ to the retrieval server, which returns matching subtrees with path annotations:

$$\texttt{retrieve}(q, M_t) = \big\{(v, \text{path}(r \rightsquigarrow v), \text{subtree}(v)) \bigm| v \in V_t,\ \text{match}(q, v)\big\}.$$

A memory update modifies the tree via $f_{\text{update}} : \mathcal{M} \times \mathcal{T} \rightarrow \mathcal{M}$.

At turn t, the agent observes input x_t and memory state $\mathcal{M}_t$, and produces a response through a trajectory of interleaved language and memory actions:

$$\tau_t = (a_{t,1}, a_{t,2}, \ldots, a_{t,L}) \sim \pi_\theta(\cdot \mid x_t, \mathcal{M}_t).$$

2.3 Verifiable Rewards

The total reward decomposes into three verifiable components:

$$R(\tau_t) = \alpha \cdot R_{\text{syntax}}(\tau_t) + \beta \cdot R_{\text{utility}}(\tau_t) + \gamma \cdot R_{\text{efficiency}}(\tau_t),$$

$R_{\text{syntax}}(\tau_t) = \mathbf{1}[\tau_t$ uses well-formed `<memory>`, `<think>`, `<answer>` tags$]$ verifies format correctness.

$R_{\text{utility}}(\tau_t) = \lambda_{\text{EM}} \cdot \mathbf{1}[y_t = y_t^*] + \lambda_{\text{sub}} \cdot \mathbf{1}[y_t^* \subseteq y_t]$ measures answer correctness against the ground truth y_t^*. For open-ended tasks, F1 or BLEU-1 may substitute.

$R_{\text{efficiency}}(\tau_t) = -\lambda_{\text{tok}} \cdot |\text{tokens}_{\text{retrieved}}|/C - \lambda_{\text{act}} \cdot |\text{mem_actions}|$ penalizes over-retrieval, incentivizing targeted subtree access.

2.4 Policy Optimization

The *policy* π_θ is a conditional distribution over action trajectories, parameterized by the LLM weights θ:

$$\pi_\theta(\tau_t \mid x_t, M_t) = \prod_{\ell=1}^{L_t} \pi_\theta(a_{t,\ell} \mid x_t, M_t, a_{t,<\ell}),$$

where $a_{t,<\ell} = (a_{t,1}, \ldots, a_{t,\ell-1})$ denotes the actions taken so far within the current turn. The policy factorizes autoregressively at the token level within each action. We decompose the system into two RL-trained agents.

Memory Manager (π_{θ_M}). After each dialogue turn, the Memory Manager observes the current state $s = (x_t, M_t^{\text{old}})$ and selects an operation to maintain the memory bank:

$$(o, m') \sim \pi_{\theta_M}(\cdot \mid x_t, M_t^{\text{old}}), \quad o \in \{\text{ADD},\ \text{UPDATE},\ \text{DELETE},\ \text{NOOP}\}.$$

The updated memory $M_t^{\text{new}} = f_{\text{update}}(M_t^{\text{old}}, o, m')$ is passed to Stage 2. The manager is rewarded based on whether its edits enable the Answer Agent to respond correctly downstream, learning to consolidate related facts (e.g., merging two pet adoption events) rather than overwrite or fragment them.

Answer Agent (π_{θ_A}). Given a user question q, the Answer Agent retrieves candidate memories $\mathcal{C} = \{m_1, \ldots, m_n\}$ via RAG, then applies a memory distillation policy to filter them:

$$\hat{\mathcal{C}} = \text{Distill}_{\pi_{\theta_A}}(\mathcal{C}, q), \qquad y = \pi_{\theta_A}(\cdot \mid q, \hat{\mathcal{C}}).$$

Distillation surfaces only the entries most relevant to q, reducing noise and improving factual accuracy. The agent is rewarded for answer correctness.

Both agents are trained with outcome-driven reinforcement learning, requiring only downstream QA correctness as supervision. For a fixed context $(x_t, \mathcal{M}_t)$, we sample a group of K trajectories and assign each a scalar reward $r(\tau_t^{(k)})$. Rather than using these rewards as absolute training signals, we optimize the policy using Group Relative Policy Optimization (GRPO), which computes a group-relative advantage for each trajectory:

$$\hat{A}^{(k)} = \frac{r(\tau_t^{(k)}) - \mu}{\sigma}, \quad \mu = \frac{1}{K}\sum_{k=1}^{K} r(\tau_t^{(k)}), \quad \sigma = \text{std}\Big(\{r(\tau_t^{(k)})\}_{k=1}^{K}\Big).$$

3 Progress to Date

3.1 System Architecture

We have implemented both Memory-R1 (flat memory) and Struct Memory-R1 (tree-structured memory) following the two-agent architecture described in Section 2. The full implementation, including the Memory Manager, Answer Agent, data construction pipeline, GRPO training scripts, evaluation metrics, and end-to-end inference pipeline, is publicly available at: `https://github.com/YifanLiu2/Struct_Memory_R1.git`.

3.2 Data Curation

We prepare data from two sources: the LoCoMo conversational benchmark (adapted for both flat and structured memory) and three structured domains from the Semantic XPath evaluation setup.

Flat LoCoMo. We use the LoCoMo benchmark [Maharana et al., 2024], a long multi-session dialogue dataset consisting of 10 conversations ($\sim$600 turns, $\sim$26k tokens each) annotated with QA pairs spanning single-hop, multi-hop, open-domain, and temporal reasoning. Following Memory-R1, we use GPT-4o-mini to extract facts from dialogue turns and build temporal flat memory banks.

Structured LoCoMo. We convert the same LoCoMo dialogues into tree-structured memory. The dataset contains: sessions of speaker turns with timestamps, per-session event summaries, per-session prose summaries, and per-session observations—speaker-attributed, turn-grounded atomic claims such as *"Caroline is researching adoption agencies"* (D2:8). We use the following schema:

```
Conversation
├── Speaker (name)
│   └── Observation (text, source_dia_id, session_id)
└── Session (session_id, datetime)
    ├── Summary (text)
    ├── Event (speaker, description)
    └── Turn (dia_id, speaker, text, image_caption)
```

`Speaker` nodes aggregate persistent observations about each participant across all sessions. `Session` nodes preserve the raw conversational record alongside two levels of derived structure: dense prose `Summary` nodes, which enable efficient session-level semantic filtering, and sparse `Event` nodes capturing key milestones. The dataset's gold observations map directly to `Observation` nodes, providing supervision for memory construction without requiring LLM-based extraction.

This schema supports two natural query patterns. *Profile queries* retrieve persistent facts about a speaker directly from their observation nodes. *Temporal queries* use `Summary` nodes as a predicate to narrow to the relevant session before retrieving specific turns within it.

Semantic XPath Domains (OOD). Following the evaluation setup in Semantic XPath [Liu et al., 2026], we use three curated structured memory domains as out-of-distribution (OOD) test sets:

- **Travel Itinerary**: Schema `Itinerary` → `Version` → `Day` → `POI` (20 QA pairs).

- **To-Do List**: Schema `TodoList` → `Category` → `Project` → `Task` (20 QA pairs).
- **Meal Kit**: Schema `MealPlan` → `Day` → `Meal` → `Option` (20 QA pairs).

These domains test generalization to unseen tree schemas and question types not present in the LoCoMo training data.

3.3 Search-R1 Baseline Verification

To validate the RL training infrastructure before adapting it for memory management, we reproduced the Search-R1 pipeline [Jin et al., 2025] on a single consumer GPU ($\geq$24 GB VRAM). This required reducing the model from Qwen2.5-7B to Qwen2.5-0.5B, lowering the GRPO group size from K=5 to K=2, enabling full FSDP CPU offloading, and training on a 100K-document Wikipedia subset (vs. 21M) indexed with FAISS. Table 1 summarizes the configuration differences.

Table 1: Search-R1 configuration: original 8-GPU vs. our single-GPU adaptation.

Parameter	**Original (8×GPU)**	**Ours (1×GPU)**
Model	Qwen2.5-7B	Qwen2.5-0.5B
Train batch size	512	8
GRPO group size K	5	2
Max prompt length	4096	2048
Max response length	500	256
vLLM GPU util.	0.6	0.3
FSDP CPU offload	partial	full
Corpus size	21M docs	100K docs

This single-GPU run confirmed that the GRPO training loop, vLLM rollout generation, reward computation, and retrieval server all function correctly end-to-end. For the Memory-R1 adaptation, we replace the search retriever with the memory bank, swap the single-agent multi-turn loop for two decoupled one-shot agents, and construct training data from LoCoMo dialogues instead of NQ question–answer pairs. The same veRL infrastructure and GRPO objective are reused throughout.

3.4 Experiment Setup

For LoCoMo, we split the QA pairs into train / validation / test following Memory-R1's 1:1:8 ratio. Memory-R1 (Flat) is trained on flat LoCoMo, and Struct Memory-R1 is trained on structured LoCoMo. The three Semantic XPath domains are used only for OOD evaluation. For the base LLM, we use Qwen2.5-0.5B, the same model used in our Search-R1 preliminary verification (Section 3.2), with the same single-GPU GRPO configuration adapted for memory operations.

3.5 Baselines and Methods

We compare six methods:

Non-RL Methods.

- **Zero-Shot (no memory)**: The base LLM answers directly without access to any external memory. This serves as a lower bound.
- **In-Context Memory**: The entire memory state is serialized and appended to the LLM prompt. Provides maximal information but scales poorly due to the "lost in the middle" phenomenon [Liu et al., 2024].
- **Flat RAG**: Standard retrieval-augmented generation with embedding-based top-k retrieval over the flat memory bank, without RL training.
- **Semantic XPath** [Liu et al., 2026]: The LLM generates an XPath-style query over the structured memory tree, executed by the Semantic XPath engine. We use results reported in the Semantic XPath paper as reference.

RL Methods.

- **Memory-R1 (Flat)** [Yan et al., 2025]: The Memory-R1 two-agent architecture operating on flat memory with ADD/UPDATE/DELETE/NOOP operations, trained with GRPO.

- **Struct Memory-R1 (ours)**: Our proposed extension of Memory-R1 to tree-structured memory, where ADD operations require a `parent_id` specifying placement in the hierarchy, trained with GRPO.

3.6 Signs of Life: Sample Trajectories

To demonstrate that our framework shows signs of life, we compare single trajectories from three methods on representative queries.

Trajectory 1: Zero-Shot (No Memory).
Q: How many pets does Alice have?
LLM: Based on my knowledge, I don't have specific information about Alice's pets. A common answer might be one or two.
Output: `**Answer:** 1`
Result: Incorrect (gold: 2 dogs). The model hallucinates without access to the conversation history.

Trajectory 2: Memory-R1 (Flat). Stage 1 – Memory Manager processes dialogue turn:
Input: Facts: ["Alice adopted two dogs named Buddy and Scout"]
Current bank: [{id:0, text:"Alice works as a software engineer"}, {id:1, text:"Alice likes cheese pizza"}]
MM output: `{''memory'': [{''id'':''0'', ''event'':''NONE''}, {''id'':''1'', ''event'':''NONE''}, {''id'':''2'', ''text'':''Alice adopted two dogs named Buddy and Scout'', ''event'':''ADD''}]}`

Stage 2 – Answer Agent:
Retrieved memories (top-3): "Alice adopted two dogs named Buddy and Scout" | "Alice works as a software engineer" | "Alice likes cheese pizza"
AA output: Selected: Memory about dog adoption.
`**Answer:** 2 dogs`
Result: Correct. The flat manager correctly ADDs the new fact; the Answer Agent distills the relevant memory.

Trajectory 3: Struct Memory-R1 (Tree). Stage 1 – Tree Memory Manager processes dialogue turn:
Current tree:
```
[Dialogue] (id=root)
[Session] (id=s1) text: Morning chat
[Memory] (id=m1) text: Alice is a software engineer
[Session] (id=s2) text: Evening catchup
[Memory] (id=m3) text: Bob is a teacher
```
Facts: ["Alice adopted two dogs named Buddy and Scout"]
MM output: `{''memory'': [{''id'':''m4'', ''parent_id'':''s1'', ''node_type'':''Memory'', ''text'':''Alice adopted two dogs: Buddy and Scout'', ''event'':''ADD''}]}`

Key: The manager places the new node under `s1` (Alice's session), not `s2` (Bob's session) – a structural decision unique to Struct Memory-R1.

Stage 2 – Answer Agent:
Retrieved memories (with paths): [Path: /Dialogue/Session(s1)/Memory(m4)] "Alice adopted two dogs: Buddy and Scout"
`**Answer:** 2 dogs`
Result: Correct. The tree path context helps disambiguate whose pets are being asked about.

3.7 Preliminary Results

Table 2 summarizes preliminary results across all evaluation datasets. For the three OOD Semantic XPath domains, we report pass rates from Liu et al. [2026]. Results for the RL-trained methods are pending completion of GRPO training.

The Semantic XPath baselines on the OOD domains achieve 80–85% pass rates with significantly fewer tokens than In-Context Memory, establishing strong non-RL reference points. Memory-R1 is expected to perform well on flat LoCoMo (its original setting), while Struct Memory-R1 targets the structured LoCoMo and OOD domains where tree-aware memory management can leverage hierarchical relationships.

Table 2: Preliminary results across evaluation datasets. LoCoMo results use Sub-EM; OOD domain results use Pass Rate. "/" indicates results pending.

Method	Flat LoCoMo	Struct LoCoMo	Itinerary	To-Do	Meal Kit
Non-RL Methods					
Zero-Shot (no memory)	/	/	15.0%	/	/
In-Context Memory	/	/	70.0%	70.0%	65.0%
Flat RAG	/	/	30.0%	/	/
Semantic XPath	–	/	85.0%	80.0%	85.0%
RL Methods					
Memory-R1 (Flat)	/	–	/	/	/
Struct Memory-R1 (ours)	–	/	/	/	/

4 The Plan Moving Forward

Our immediate next step is to complete GRPO training for both Memory-R1 (flat) and Struct Memory-R1 (tree) using Qwen2.5-0.5B as the base model. Specifically, we plan to:

1. Run the two-stage training pipeline (Memory Manager → Answer Agent) on the LoCoMo training split for both flat and structured memory configurations.
2. Evaluate on all five datasets (flat LoCoMo, structured LoCoMo, and the three OOD domains) using F1, BLEU-1, Sub-EM, and LLM-as-a-Judge metrics.
3. Analyze the results to understand the effect of structured vs. flat memory on different question types (single-hop, multi-hop, temporal, open-domain) and on in-distribution vs. OOD generalization.

References

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

Yifan Simon Liu, Ruifan Wu, Liam Gallagher, Jiazhou Liang, Armin Toroghi, and Scott Sanner. Semantic XPath: Structured agentic memory access for conversational AI. *Under Review*, 2026.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.

Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Kristian Kersting, Jeff Z Pan, Hinrich Schütze, et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.